SIMATS ENGINEERING

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 20%

Code of Conduct:

*I **SANTHOSH R (192321007)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

SANTHOSH R

Signature of the student:

CO2_AT3 Error Analysis Task: Software Testing (CSA37)

Answer 1: Equivalence Partitioning and Boundary Value Analysis (Online Shopping)

Assumption: the field under test is the "Quantity" input of the cart, which accepts whole numbers from **1 to 10** per item.

1. Errors identified

The tester's flawed design uses two classes: Class A = {−5, 0, 1, 5} and Class B = {10, 11, 50}, with one representative picked from each.

- **Mixed classes:** Class A combines invalid values (−5, 0) with valid values (1, 5). Class B combines a valid value (10) with invalid values (11, 50). Members of a class must all be treated the same way by the system, and these are not.
- **Incomplete partitions:** no class exists for non-integer input (2.5), non-numeric input ("abc", special characters) or an empty field.
- **No boundary values:** 0, 1, 10 and 11 are never tested as deliberate boundary points; they appear only by accident.
- **Invalid-class rule violated:** each invalid condition should have its own class so that one failure cannot mask another.

2. Root causes

- Partitions were built from the data's appearance ("small numbers", "large numbers") instead of from the specification's rule "valid = 1 to 10".
- The tester did not separate valid from invalid input domains, which is the basic principle of equivalence partitioning.
- Boundary value analysis was treated as optional, although most off-by-one defects (using < instead of <=) occur exactly at the edges.
- There was no review or traceability check between the requirement and the test design.

3. Corrected design

Class	Description	Range / Example	Type
EC1	Valid quantity	1 to 10 (e.g., 5)	Valid
EC2	Below minimum (zero / negative)	≤ 0 (e.g., −3)	Invalid
EC3	Above maximum	≥ 11 (e.g., 50)	Invalid
EC4	Non-integer (decimal)	e.g., 2.5	Invalid
EC5	Non-numeric	e.g., "abc", "@#"	Invalid
EC6	Empty / null	(blank)	Invalid

TC ID	Input	Class / Technique	Expected result
TC1	5	EC1 (ECP)	Accepted, added to cart
TC2	0	Boundary (min − 1), EC2	Rejected, error message
TC3	1	Boundary (min)	Accepted
TC4	2	Boundary (min + 1)	Accepted
TC5	9	Boundary (max − 1)	Accepted
TC6	10	Boundary (max)	Accepted
TC7	11	Boundary (max + 1), EC3	Rejected, error message
TC8	−3	EC2 (ECP)	Rejected, error message
TC9	2.5	EC4 (ECP)	Rejected, "enter whole number"
TC10	abc	EC5 (ECP)	Rejected, "enter a number"
TC11	(blank)	EC6 (ECP)	Rejected, "quantity required"

4. Justification

- Each class is homogeneous, so one test per class gives reliable coverage with few tests (11 instead of an unbounded set).
- Separate invalid classes mean every error-handling path is exercised and a failure points to one specific cause.
- The six boundary tests (0, 1, 2, 9, 10, 11) target the places where programmers most often make relational-operator errors, so defect detection is much higher than with arbitrary values.

Answer 2: Random Testing of a Banking Application

Assumption: a transaction must be between ₹100 and ₹50,000, and the daily withdrawal limit is ₹1,00,000.

1. Errors in strategy and coverage assumptions

- **Unsupported coverage claim:** "high coverage" was stated without a coverage measure (requirements, partitions, statements or branches). The number of tests run is not coverage.
- **No structured technique:** random inputs have no link to the specification, so there is no traceability and no way to show which requirements were checked.
- **Boundaries ignored:** limit values such as 99, 100, 50,000 and 50,001 are single points in a very large input space; random selection almost never hits them.
- **Weak oracle and no repeatability:** without expected results fixed in advance, wrong outputs can be accepted, and failed runs cannot be reproduced.
- **Combinations and states missed:** account status (active, frozen), balance versus limit and daily totals were not considered.

2. Root causes of the missed defects

- Defects in limits are triggered by specific values. If a range has 50,000 possible values, the chance that a random test picks one given boundary is about 1 in 50,000.
- Random data clusters in the middle of ranges, which are the values most likely to work correctly.
- The engineer relied on volume instead of design, and had no test adequacy criterion to decide when testing was complete.

3. Corrected approach: suitable black-box techniques

Technique	Application to banking	Example
Equivalence Partitioning	Split amount into valid (100–50,000) and invalid (<100, >50,000, non-numeric, negative)	Test 25,000; 50; 75,000; −500; "abc"
Boundary Value Analysis	Test at and around every limit	99, 100, 101, 49,999, 50,000, 50,001
Decision Table Testing	Combine balance sufficient, within limit, account active	8 rules, each with expected outcome
State Transition Testing	Account states: Active, Dormant, Frozen, Closed	Withdraw from a Frozen account must fail
Error Guessing (supplement)	Experience-based extras after structured tests	Duplicate submit, timeout during transfer

4. Justification

- **Adequacy:** coverage can now be measured and reported (all partitions, all boundaries, all decision-table rules, all transitions covered), so "done" has a clear meaning.
- **Reliability:** every test has fixed inputs and expected results, is traceable to a requirement and can be repeated, so defects found can be confirmed and fixed defects re-tested.
- **Efficiency:** about 15 to 25 designed tests cover the transaction-limit logic that thousands of random tests missed.

Answer 3: Control Flow Graph and Path Testing

The following module is used to illustrate the analysis:

```
int process(int n, int a[]) {
1     int sum = 0, i = 0, neg = 0;
2     while (i < n) {
3         if (a[i] > 0)
4             sum += a[i];
5         else if (a[i] < 0)
6             neg++;
7         i++;
      }
8     if (sum > 100)
9         return 1;
10    return 0;
}
```

1. Errors in the developer's CFG

- The **while** decision (node 2) is drawn as a plain statement, and the **loop-back edge** from node 7 to node 2 is missing, so iteration is not represented.
- The **else-if** decision (node 5) and its two outgoing edges are missing, and the false branch of node 3 is not connected to anything.
- The two exits (nodes 9 and 10) are not merged into one exit node.
- The incomplete graph has only 2 decision nodes, so cyclomatic complexity is calculated as V(G) = 3, whereas the real value is 5.

2. Root causes

- The graph was drawn from a quick reading of the code rather than by systematically converting every control construct (if, else-if, while, return).
- Compound constructs (an else-if chain and a loop) were treated as simple statements; the developer did not know that each predicate becomes a decision node.
- No cross-check using V(G) = E − N + 2 or V(G) = P + 1 was done; it would have exposed the gap.
- With a wrong graph, the derived paths are too few, so test cases cover only a subset of the program's logic.

3. Corrected CFG

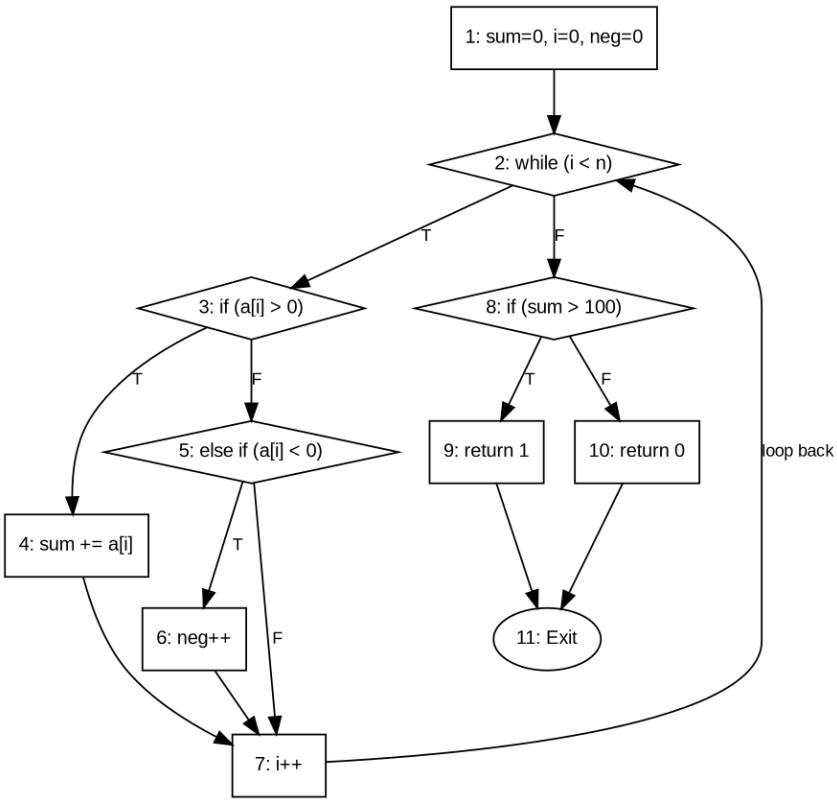


Figure 1: Corrected CFG (N = 11 nodes, E = 14 edges, 4 predicate nodes: 2, 3, 5, 8)

Cyclomatic complexity: V(G) = E − N + 2 = 14 − 11 + 2 = **5**, and V(G) = P + 1 = 4 + 1 = **5**, so there are 5 independent paths.

Path	Node sequence	Test data	Expected result
P1	1-2-8-10-11	n = 0	returns 0 (loop skipped)
P2	1-2-3-4-7-2-8-10-11	n = 1, a = {5}	returns 0 (sum = 5)
P3	1-2-3-5-6-7-2-8-10-11	n = 1, a = {−3}	returns 0, neg = 1
P4	1-2-3-5-7-2-8-10-11	n = 1, a = {0}	returns 0, no change
P5	1-2-3-4-7-2-8-9-11	n = 1, a = {150}	returns 1 (sum > 100)

4. Justification

- The basis set of 5 paths covers every edge of the graph at least once, which gives full statement and branch coverage and exercises the loop zero times and more than zero times.
- Because the basis set is derived from the complete graph, any other path is a combination of these, so a logic defect on a decision or loop edge will be exposed.
- V(G) also gives a measurable target: if fewer than 5 tests are designed for this module, coverage is provably incomplete.

Answer 4: Statement Coverage versus Logical Errors

```
1  total = price * qty;
2  if (total > 1000)          /* specification: total >= 1000 */
3      total = total * 0.90;
4  return total;
```

1. Evaluation and gaps in coverage criteria

One test, price = 500 and qty = 4 (total = 2000), executes lines 1 to 4, so **statement coverage is 100%**. However:

- The **false branch** of line 2 is never executed, so branch coverage is only 50%.
- The **relational-operator defect** (> written instead of >=) is not found, because the boundary total = 1000 is never tested.
- The case where the condition is false and no discount applies (total = 500) is never checked.
- For compound conditions such as (A && B), statement coverage does not show whether each condition affects the outcome.

2. Why statement coverage alone was insufficient

- It only proves that every line was run at least once, not that every decision outcome or condition combination was run.
- An if statement with no else has an implicit empty branch that contains no statement, so it can be skipped without lowering the coverage figure.
- Execution of a statement does not mean its result was verified; coverage measures what ran, not what was checked.

3. Recommended white-box strategy

Test	Input total	Decision outcome	Expected total	Purpose
T1	2000	True	1800	Statement + true branch
T2	500	False	500	False branch
T3	1000	True (per spec)	900	Exposes > vs >= defect
T4	999	False	999	Boundary below

T3 fails on the faulty code (it returns 1000), so the defect is detected. For compound predicates, use **condition coverage** and **MC/DC**; for loops and multiple decisions, use **path coverage** (basis path testing).

4. Justification

- The coverage criteria form a hierarchy: path coverage includes MC/DC, which includes branch coverage, which includes statement coverage. Moving up the hierarchy never reduces the defects found.
- Branch coverage forces both outcomes of every decision, which removes the blind spot of untested false branches.
- Combined with boundary tests (T3, T4), the approach finds both structural and logical errors in conditionals.

Answer 5: Equivalence Partitioning for Age Validation (0 to 100)

1. Errors in the partitioning strategy

- Only two classes (Negative and Positive) were defined, but the specification has **three numeric regions**: below 0, 0 to 100, above 100.
- "Positive" mixes valid ages (1 to 100) with invalid ages (101 and above), so the class is not homogeneous.
- The value **0** belongs to neither class, although it is a valid age and a lower boundary.
- Non-numeric, decimal and blank inputs are not considered.
- No boundary value analysis is present, so −1, 0, 100 and 101 are never checked.

2. Why it fails to separate valid from invalid limits

The classes were created using the sign of the number (mathematical property) and not using the validation rule in the requirement. The sign changes at 0, but the valid range ends at 100, which the tester's classes cannot represent. A 150-year-old "age" would be treated as acceptable by this test design.

3. Corrected partitions

Class	Description	Range / Example	Type	Expected
EC1	Valid age	0 to 100 (e.g., 35)	Valid	Accepted
EC2	Below minimum	< 0 (e.g., −5)	Invalid	Rejected
EC3	Above maximum	> 100 (e.g., 150)	Invalid	Rejected

Class	Description	Range / Example	Type	Expected
EC4	Decimal	e.g., 25.5	Invalid	Rejected
EC5	Non-numeric	e.g., “abc”	Invalid	Rejected
EC6	Empty	(blank)	Invalid	Rejected

BVA test	−1	0	1	99	100	101
Expected	Reject	Accept	Accept	Accept	Accept	Reject

4. Justification

- Valid and invalid inputs are now in separate classes, so every rule of the validation is exercised independently.
- BVA tests at −1/0 and 100/101 catch the most common defects (< vs <=, off-by-one) at the exact limits.
- The input validation becomes robust against negative, over-range, decimal, text and empty entries with only 12 tests.

Answer 6: Condition Coverage for a Complex Decision

Note: the text of Question 6 is cut off in the question paper after “if(A && B”. The answer below assumes the decision **if ((A && B) || C)**; the same method applies to any compound condition.

1. Errors in the developer’s claim

Suppose the developer ran two tests: T1 (A = T, B = T, C = F) and T2 (A = F, B = F, C = T).

Test	A	B	C	(A && B) || C
T1	T	T	F	True
T2	F	F	T	True

- Each of A, B and C has taken both True and False, so **condition coverage = 100%**.
- But the decision is **True in both tests**; the False outcome is never run, so **decision (branch) coverage is only 50%**.
- Condition coverage does not show that each condition independently affects the outcome; for example, B was never shown to change the result alone.

2. Root causes

- Confusing condition coverage (each atomic condition) with decision coverage (the overall result) and assuming that one implies the other. It does not.
- Choosing values to make each condition toggle without computing the resulting decision value.
- Ignoring short-circuit evaluation: in A && B, B is not evaluated when A is False, so some conditions are never actually exercised.

3. Corrected test set: MC/DC

For n = 3 conditions, MC/DC needs n + 1 = **4 tests**, where each condition has a pair of tests that differ only in that condition and give different outcomes.

Test	A	B	C	Outcome	Shows independence of
M1	T	T	F	True	A (with M2), B (with M3)
M2	F	T	F	False	A
M3	T	F	F	False	B (with M1), C (with M4)
M4	T	F	T	True	C

4. Justification

- The set gives both decision outcomes (True and False), both values of every condition, and proof that each condition alone can change the result.
- It needs only n + 1 tests instead of $2^n = 8$ for exhaustive testing, so cost stays linear while fault-detection power (relational and logical-operator faults) stays high.

- MC/DC is the criterion required for safety-critical software (for example DO-178C Level A), which shows its reliability.

Answer 7: Decision Table Testing for a Discount System

Assumed business rules: a Premium Member gets 10% off; a valid Coupon gives 5% off; the two discounts add up; an invalid or expired coupon is rejected with a message.

1. Errors in the decision table construction

- Not all combinations of conditions were listed. With 3 conditions there must be $2^3 = 8$ rules; the incomplete table has fewer, so combinations like "Premium + Coupon applied" and "Premium + invalid coupon" are absent.
- The third condition (is the coupon valid?) was not identified, so the invalid-coupon behaviour is never tested.
- The interaction of the discounts (stacking) was not stated as an action, so the combined price calculation was never verified.

2. Why the current approach is incomplete

The table was built from the conditions that came to mind, instead of enumerating all True/False combinations. A table with missing columns hides specific rule combinations, and these are exactly the cases where price calculation errors (double discount, wrong order of discounts, discount applied on rejected coupon) occur.

3. Revised exhaustive decision table

Conditions / Actions	R1	R2	R3	R4	R5	R6	R7	R8
C1: Premium Member	Y	Y	Y	Y	N	N	N	N
C2: Coupon applied	Y	Y	N	N	Y	Y	N	N
C3: Coupon valid	Y	N	Y	N	Y	N	Y	N
A1: Premium discount 10%	X	X	X	X				
A2: Coupon discount 5%	X				X			
A3: Show "invalid coupon"		X				X		
Total discount	15%	10%	10%	10%	5%	0%	0%	0%

Rules R3/R4 and R7/R8 give the same outcome because C3 is irrelevant when no coupon is applied; they can be merged to 6 test cases if desired, but all 8 are shown to prove completeness. Example, for a cart of ₹1,000: R1 = ₹850, R2 = ₹900, R5 = ₹950, R8 = ₹1,000.

4. Effectiveness in preventing production defects

- Every condition combination has a defined expected result, so no combination reaches production untested.
- Gaps and contradictions in the requirements are found while building the table, before coding.
- It is easy to review with business users and to update if a rule changes (add a condition, double the columns).

Answer 8: Loop Testing

Assumption: the loop is for (i = 0; i < n; i++), where n can be at most max = 100.

1. Errors in the tester's loop strategy

- Only n = 1 and n = max were tested, so the **zero-iteration** case (n = 0) is skipped; a defect here (e.g., uninitialised result or do-while behaviour) is missed.
- **n = max + 1** (one iteration beyond the limit) is not tested, so buffer overflow, array index out of bounds and off-by-one errors in the loop condition are missed.
- Typical values (2, a middle value) and max − 1 are not tested, so errors that appear only on subsequent iterations are missed.
- Nested loops and the effect of the loop body on the loop counter are not considered.

2. Root causes

- The tester assumed that the first and last iterations represent all iterations, which ignores the fact that defects occur at boundary counts of the loop.
- The tester viewed the loop as a single statement and not as a decision that is re-evaluated before every iteration.
- Specified limits were not turned into boundary conditions (min − 1, min, max, max + 1).

3. Corrected approach

Simple loop (maximum m = 100 passes):

Test	Iterations	Purpose
L1	0	Loop skipped entirely
L2	1	Minimum pass
L3	2	Repeat behaviour starts
L4	m (e.g., 50)	Typical value
L5	max − 1 (99)	Just below limit
L6	max (100)	At limit
L7	max + 1 (101)	Beyond limit: must be rejected or handled

Nested loops (Beizer's method):

- Start with the innermost loop; set all outer loops to their minimum iteration count.
- Test the innermost loop as a simple loop (0, 1, 2, typical, max − 1, max, max + 1) while outer loops stay at minimum.
- Move outward one level at a time, keeping the inner loops at typical values.
- Finally, run the test with all loops at their maximum values.
- For concatenated loops that share a counter, treat them as nested; for unstructured loops, redesign the code first.

4. Justification

- The six-to-seven-test pattern exercises the zero, one, many, limit and over-limit cases, so every possible path through the loop is checked at least once at its critical counts.
- The nested approach reduces tests from $(max)^n$ to a number that grows linearly with the nesting depth, while still covering path integrity.

Answer 9: Error Guessing for a Safety-Critical Braking System

1. Errors in relying on ad-hoc testing

- **Dependent on the tester's experience:** error guessing finds only the faults a person imagines, so unforeseen faults remain.
- **No systematic coverage:** there is no measure showing which input ranges, states and transitions were tested.
- **Not repeatable or traceable:** results cannot be audited against requirements, which safety standards such as ISO 26262 demand for high-integrity (ASIL C/D) functions.
- **Structured black-box techniques were ignored,** so boundary conditions (sensor limits, thresholds) and state-dependent behaviour are not verified.

2. Impact on test adequacy

Without a defined adequacy criterion, testing cannot be shown to be complete. Critical faults, for example a threshold off-by-one or a wrong state change after a sensor failure, may stay undetected until an actual braking event. In a safety-critical system, the consequence of an escaped defect is loss of life, so low and unmeasurable adequacy is unacceptable.

3. Corrected strategy: BVA + State Transition Testing

(a) Boundary Value Analysis (assumed: pedal pressure 0 to 100%, ABS threshold: wheel slip > 15%)

Parameter	Boundary test values	Expected behaviour
Pedal pressure (%)	−1, 0, 1, 99, 100, 101	−1 and 101: sensor error flagged; 0: no braking; 1 to 100: proportional braking
Wheel slip (%)	14, 15, 16	14, 15: normal braking; 16: ABS activates

(b) State Transition Testing

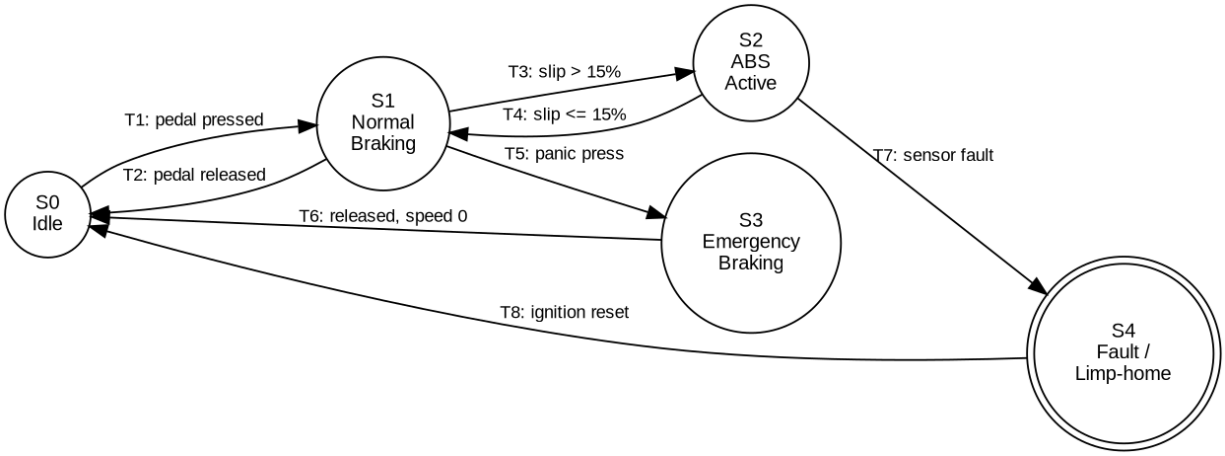


Figure 2: State transition diagram of the braking controller

Test	Sequence	Expected result
ST1	S0 → S1 (T1)	Braking begins when pedal is pressed
ST2	S1 → S0 (T2)	Brakes released
ST3	S1 → S2 → S1 (T3, T4)	ABS engages and then disengages
ST4	S1 → S3 → S0 (T5, T6)	Full emergency force, then release
ST5	S2 → S4 → S0 (T7, T8)	Fault detected, limp-home braking, reset
ST6 (invalid)	S0 + slip event	No action; ignored safely
ST7 (invalid)	S4 + pedal pressed	Backup braking only; ABS not activated

4. Justification of increased safety assurance

- BVA shows that every threshold and sensor limit behaves correctly at, just below and just above its value.
- State transition testing proves that every valid transition happens and every invalid event is safely refused, including fault handling.
- Both are traceable and repeatable, which supports certification evidence; error guessing is kept only as an additional technique on top of them.

Answer 10: Missing “else” Branches in a Control Flow Graph

```
read a, b, c;
if (a > 0)
    if (b > 0)
        if (c > 0)
            result = a + b + c;
        else error = 3;
    else error = 2;
else error = 1;
output;
```

1. Errors in the graph construction

- The tester's CFG contains only the "true" chain: nodes 1, 2, 3, 4, 5 and the exit. The three **else** nodes (error = 3, 2, 1) and their edges are absent.
- Of the 6 decision edges (3 true + 3 false), only the 3 true edges exist in the graph.
- One test (a, b, c all > 0) executes every node of that incomplete graph, giving the "100% statement coverage", while the three error paths are never run: **0% coverage of the error branches**.

2. Root causes

- The graph was drawn from the normal (happy-path) flow only; implicit and explicit else branches were treated as unimportant.
- Nested ifs were flattened, so the nesting depth and the decision at each level were lost.
- Coverage was computed on the incomplete model and not on the real code, which gave a false sense of adequacy.
- No check of V(G) = P + 1 was performed. With 3 predicates V(G) must be 4; the tester's graph gave 1.

3. Corrected CFG and independent paths

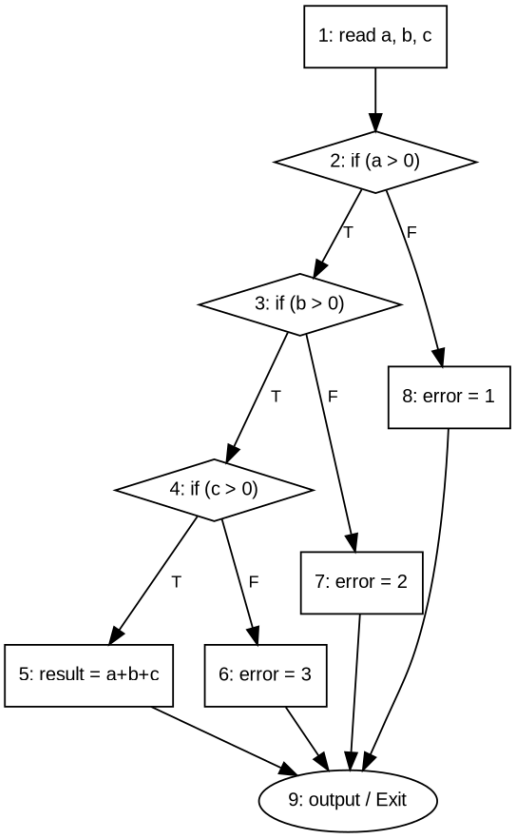


Figure 3: Corrected CFG (N = 9, E = 11, P = 3)

V(G) = E − N + 2 = 11 − 9 + 2 = **4** = P + 1. So there are 4 independent paths:

Path	Node sequence	Test data (a, b, c)	Expected result
P1	1-2-3-4-5-9	(1, 1, 1)	result = 3
P2	1-2-3-4-6-9	(1, 1, 0)	error = 3
P3	1-2-3-7-9	(1, 0, 1)	error = 2
P4	1-2-8-9	(0, 1, 1)	error = 1

4. Branch adequacy

- The four tests cover all 11 edges, including all 3 false edges, so statement coverage and branch coverage are both 100%.
- Every error-handling path is now executed, and each is verified against its expected result.

- Rule applied: draw an edge for every outcome of every decision (including implicit else), and compute coverage only on the complete graph.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand